AXIOM: Correctness-Aware Multi-Agent Slice Admission Control with Federated Training

Manoj Prasad Kunasegran[1], Wai Leong Pang[1*], Swee King Phang[1]

[1]School of Engineering, Taylor's University, Subang Jaya, Malaysia.

*Corresponding author(s). E-mail(s): waileong.pang@taylors.edu.my;

Abstract

Fifth generation (5G) network slicing must admit heterogeneous service requests across many cooperating base stations under strict per-slice guarantees. This paper presents AXIOM, a multi-agent slice admission controller with a shared cluster encoder and decentralised per-slice action heads, together with a pre-registered evaluation protocol designed to make its results trustworthy. We test it in two simulators, a static-load shared-budget environment and a harder one with time-varying load and inter-cell coupling, against single-agent and independent deep Q-network (DQN) baselines, multi-agent proximal policy optimisation (MAPPO), and tuned threshold rules. In the static-load environment AXIOM comes within 1.1% of an oracle rule's reward at three gNodeBs (gNBs). At 7 and 19 gNBs it keeps Ultra-Reliable Low-Latency Communication (URLLC) and enhanced Mobile Broadband (eMBB) Service Level Agreement (SLA) compliance near 97%, against 56% to 76% for every baseline, with all 36 per-layout comparisons significant over 15 to 30 seeds. In the coupled environment AXIOM beats the static and accept-all rules by 39% and 103% in reward and leads every learned baseline at three gNBs. A three-parameter rule tuned with domain knowledge still outperforms every learned policy there, however, and at 7 gNBs AXIOM does no better than the DQN baselines. A per-node encoder matches or beats graph attention throughout, so AXIOM's gains come from its shared encoder, per-slice heads, and joint training. Federated training shows no significant cost, per-step differential privacy gives only a weak formal guarantee, and AXIOM's parameter count stays at 7,782 at any cluster size. A trivial-policy check, per-slice correctness metrics, independent-seed replication, and pilot checks caught seven evaluation failures, including two simulator flaws. A live OpenAirInterface (OAI) testbed validates the control path AXIOM would use on a real Open Radio Access Network (O-RAN) stack, while AXIOM itself is evaluated offline.

Keywords: Network Slicing, Slice Admission Control, Multi-Agent Reinforcement Learning, Graph Attention Networks, Federated Learning, Open RAN

M. P. Kunasegran: orcid.org/0009-0005-3169-1873. W. L. Pang: orcid.org/0000-0001-8407-5648. S. K. Phang: orcid.org/0000-0002-7877-8766.

1 Introduction

Network slicing virtualises the physical radio access network (RAN) to serve heterogeneous services under Ultra-Reliable Low-Latency Communication (URLLC), enhanced Mobile Broadband (eMBB), and massive Machine-Type Communication (mMTC) categories. Each slice carries distinct Service Level Agreement (SLA) targets that are in turn based on Quality of Service (QoS) thresholds. Our earlier work [13, 14] validated that a Deep Q-Network (DQN) admission controller can enforce SLAs for a single gNodeB (gNB) in a live OpenAirInterface (OAI) testbed. However, a real deployment spans multiple gNBs that share backhaul, spectrum planning, and a cluster-wide load-balance objective. Thus, slice admission at one gNB is rarely isolated, and coordinated admission is in fact a multi-gNB, multi-agent challenge. This research aims to address whether a shared, graph-structured view of the cluster could enable per-gNB agents to decide better than a single agent on a flattened state, as opposed to independent agents. Fig. 1 places the research premise in context.

This paper introduces AXIOM, an Attention-based eXecution with Integrity for O-RAN Multi-agent slice-admission controller. AXIOM is a centralised-training, decentralised-execution multi-agent architecture that is built on a shared Graph Attention Network (GAT) [29] encoder over a multi-gNB topology, which feeds decentralised per-slice Q-heads. We deliver it together with the evaluation discipline needed to trust what it reports. Three gaps from our systematic review of 67 core studies motivate this pairing, as Section 2 details. First, no reviewed study checks whether its own outcome metric can score a degenerate policy as well as a correct one. Second, none replicates a headline architectural finding on an independent seed sample. Third, none tests whether a live testbed can reach the contention regime its offline claims

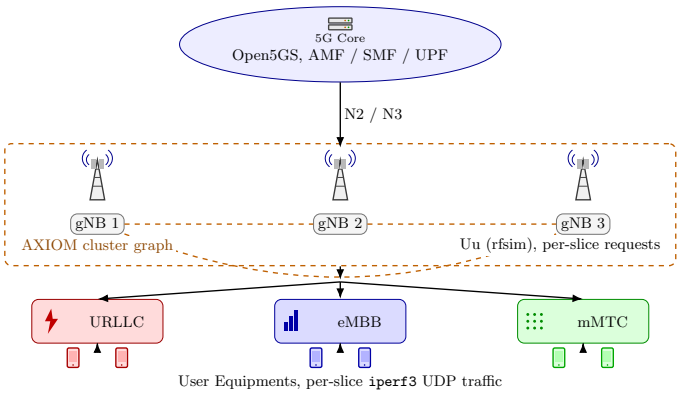


Fig. 1 Conceptual system view. A cluster of gNBs, coordinated by the shared AXIOM encoder and backhauled to a 5G core, serves UEs across three slices. AXIOM is evaluated offline; the live rig runs one gNB.

assume. We address these gaps through a per-component realisation of the architecture, namely the GAT encoder, the Multi-Agent Reinforcement Learning (MARL) controller, the Federated Learning (FL) coordinator, and the Differential Privacy (DP) layer, each measured and isolated rather than folded into one end-to-end number. We apply a correctness-aware metric pair and a reproduction-versus-replication discipline throughout, and we add a live single-gNB deployment that tests, rather than assumes, whether the rig reaches genuine per-slice scarcity.

We evaluate AXIOM in two offline simulators of a gNB cluster. The first, Env A, has a static load in which shedding every mMTC request is optimal. The second, Env B, adds time-varying load and inter-cell coupling, so that no fixed rule is optimal and neighbouring gNBs affect each other. We compare AXIOM with single-agent and independent deep Q-networks, with multi-agent proximal policy optimisation (MAPPO), and with tuned threshold rules. We also extend it to federated training with differential privacy, stress it under gNB dropout, demand spikes, and agent churn, and grow the cluster to 7 and 19 gNBs under fully-connected, ring, and hex-grid adjacency. The contributions of this paper are as follows.

- An evaluation protocol for learned slice admission control that combines trivial-policy checks, per-slice correctness metrics, independent-seed replication, tuned rule baselines, and a pre-registered analysis, and that caught seven evaluation failures in this work, including two simulator flaws.
- AXIOM, a multi-agent controller with a shared cluster encoder and decentralised per-slice Q-heads, whose parameter count does not grow with the cluster, and which keeps priority-slice compliance near 97% at 7 and 19 gNBs in Env A, against 56% to 76% for both DQN baselines and MAPPO.
- A harder environment with time-varying load and inter-cell coupling, in which learned policies beat static rules but a tuned threshold rule still beats them all, a result we report rather than tune away.
- Negative results that bound the architecture's claims, namely that graph attention adds nothing measurable over a per-node encoder, that per-step differential privacy gives no meaningful guarantee, and that the live testbed validates only the control path.

2 Related Work

Our prior systematic review [15] screened 6,286 records down to 67 core studies from 2024 to 2026 on machine-learning-driven, user-centric distributed network slicing. The review organised them around three enabling pillars, namely topology-aware graph representation, distributed multi-agent control, and privacy-preserving federation. Each pillar is well developed on its own, but the three are rarely combined under realistic constraints, and never with a validated live deployment. No reviewed study checks whether its own evaluation metric can mask a degenerate policy, and none replicates a headline finding on an independent seed sample, which is the gap this paper fills.

AXIOM also sits inside the broader Open RAN (O-RAN) effort to run learned control loops as applications over standard interfaces. Polese et al. [20] survey that

architecture and its open problems, while ColO-RAN [19] and Tsampazi et al. [28] train and compare deep reinforcement learning controllers for slicing on large experimental platforms. Learned slice admission predates O-RAN, from market-driven admission of slice requests [4] to reinforcement learning for RAN slicing [23]. For cooperating agents, value factorisation such as QMIX [22] and centralised-critic policy gradients such as MAPPO [31] are the standard training schemes, and we compare against MAPPO directly. Constrained multi-agent learning has also been applied to Open RAN slicing at scale [32], and Khani et al. [10] survey deep reinforcement learning for slice admission in cloud RAN. Within this journal, recent work applies actor-critic learning to joint resource allocation in RAN slicing [12], double deep Q-networks to next-generation networks [9], and delay-aware slice allocation to cloud RAN [3]. Other work in the journal applies federated learning to 6G slice resource allocation [11] and multi-agent reinforcement learning to device-to-device relay selection [25]. None of these evaluates against a trivial-policy check or replicates its headline result on an independent seed sample. AXIOM instead shares one graph encoder across agents and trains every pending request as its own temporal-difference sample, for the reason Section 4.1 gives.

Several recent works come close on a single axis of this problem. Shao et al. [24] motivate our GAT encoder choice by showing that graph attention strengthens inter-agent cooperation over plain DQN and Advantage Actor-Critic (A2C) baselines. Ahmadi et al. [2] study topology generalisation through graph attention, which complements but does not answer the fixed-topology evaluation-integrity question we ask in Section 5.6. Sulaiman et al. [26] are the closest prior art to our joint slicing-and-admission-control problem. None of these three checks whether its reward-optimal policy could be mistaken for a degenerate one, and none replicates a headline result on an independent seed sample.

Robustness studies share our concern but stop short of it. Tashman and Cherkaoui [27] and Fatehi et al. [7] study robustness to a disruptive condition, as our disruption campaign in Section 5.5 does, yet neither checks whether the reported robustness metric is trustworthy and neither replicates a finding on an independent sample. That is the gap our independent-seed replication closes, and the need is concrete, since replication overturned an early churn-immunity reading in our earlier simulator. Zhang et al. [33] are the closest periodic-synchronisation precedent, and Yasin et al. [30] are the closest precedent for federated learning with differential privacy. Their threat model differs from ours, since they secure the controller rather than characterise a control policy's own privacy-utility trade-off, which our σ sweep in Section 5.4 reports, and neither adds a correctness-aware check on whether federating or privatising changed decision quality. A further, closely related set [5, 8, 18, 21] combines two of the three pillars, yet like every work above, each evaluates on its own reported outcome metric, computed once, on one seed or run.

Rather than adding a new point in the GAT, MARL, and FL design space, this paper fills that gap with a demonstration on AXIOM, extended to federation, privacy, and disruption. It shows that two checks are both necessary, namely testing a metric's own trustworthiness and replicating a finding on an independent sample. Skipping either changes what a reader may believe the resulting numbers mean.

3 System Model and Problem Formulation

Fig. 2 provides an overview of the system. The AXIOM controller is evaluated offline in two shared-budget simulation environments, Env A and Env B. Evaluation safeguards and a pre-registered analysis then check every reported result. The deployment pipeline is then checked on the live single-gNB testbed of Fig. 3.

We consider a cluster of N=3 gNBs, each serving the same three slices, URLLC, eMBB, and mMTC, under a shared per-gNB Physical Resource Block (PRB) budget B. An admission decision is a PRB-ceiling adjustment through the `slicing_control_m` minimum and maximum ratio, rather than a literal per-flow accept or reject, since the real OAI xApp control interface exposes only ratio ceilings. No real multi-gNB physical topology is available to us, so we take the least assumption-laden choice and treat the cluster as fully connected, as a modelling choice rather than a measured fact. At each control step, every gNB holds zero or more pending admission requests, each tagged with a slice identity, and an agent decides whether to accept or reject for each request independently, conditioned on the current cluster state. The discrete action space matches our single-gNB work exactly at `action_dim`=2, so any coordination benefit found here is due to the architecture and not a richer action space.

The step reward follows our prior single-gNB formulation. It has a per-slice service term net of SLA-violation penalties, a congestion charge on admitted volume, and a load-imbalance penalty across gNBs. The step reward r_t sums over slices indexed by k. It uses priority weight ω_k, per-accept reward R_k, admitted count n_k, violation penalty λ_k, and the indicator $\mathbf{1}[\cdot]$ equal to one exactly when slice k is in SLA violation that step,

$$r_t = \sum_k \big(\omega_k R_k\, n_k - \lambda_k\, \mathbf{1}[v_k]\big) \;-\; \kappa\, C_t \|a_t\|_1 \;-\; \beta\,(1 - F_t), \tag{1}$$

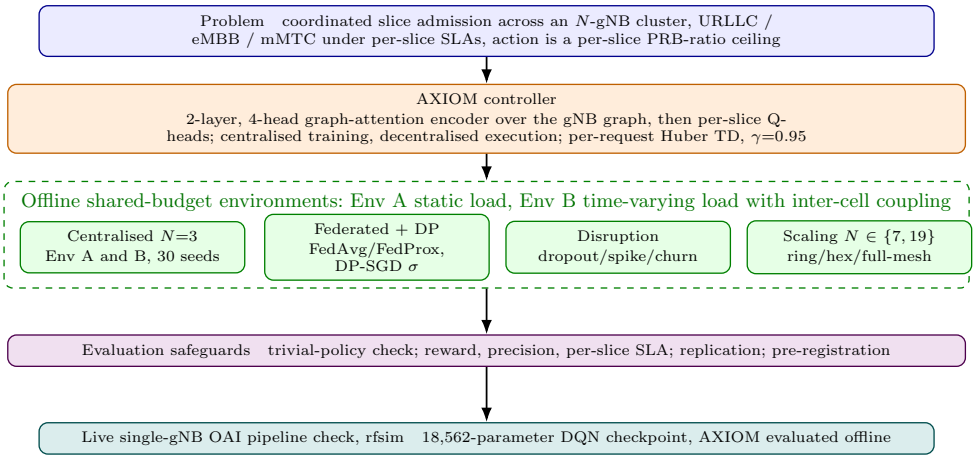


Fig. 2 Overview of the research. An AXIOM controller is evaluated across four offline campaigns in two shared-budget environments, every result checked by evaluation safeguards, with the deployment pipeline checked on a live single-gNB testbed.

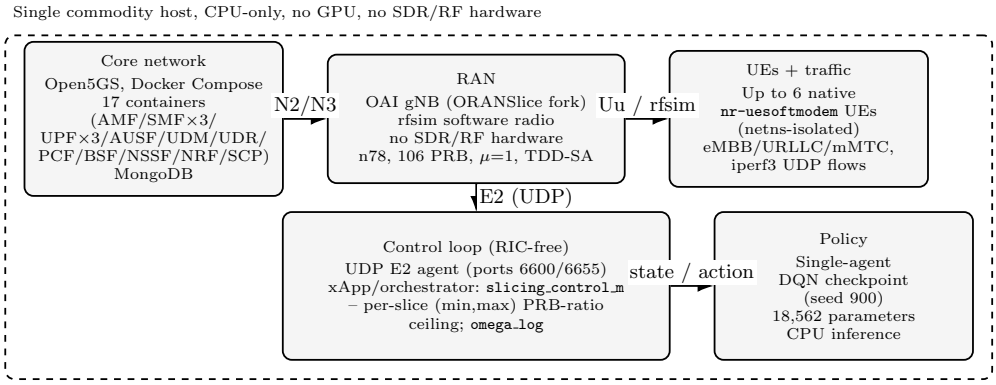


Fig. 3 Real testbed stack, a single-host, software-radio rfsim, RIC-free O-RAN deployment distinguished from SDR hardware-in-the-loop testbeds. Every block and label is a measured or configured value from this rig.

where $\|a_t\|_1$ is the total admitted this step, C_t the cluster-mean congestion level, κ the congestion coefficient, $F_t \in [0,1]$ the min-over-max PRB-utilisation fairness ratio, and β the load-balance weight. The violation flag v_k follows from a continuous SLA margin m_k. The correctness-aware metrics of Section 4.4 also build on this margin. We take it as the worse of the queue and loss channels,

$$m_k = \min\Big(1 - \tilde{q}_k,\ 1 - \tfrac{\ell_k}{\ell_k^{\max}}\Big), \qquad v_k \Leftrightarrow m_k \le 0, \tag{2}$$

with $\tilde{q}_k$ the slice backlog relative to its budget, and ℓ_k and $\ell_k^{\max}$ its loss proxy and loss budget. The weights are set so that, under congestion, the intended policy is differentiated shedding, which keeps accepting URLLC and eMBB and sheds mMTC.

Every result uses an offline, closed-loop simulation of the cluster, except the bounded live anchor of Section 5.9. Each gNB has a budget of B=100 PRBs shared by its three slices. Each slice's mean offered demand is 1.05 times its nominal PRB share of 30, 60, and 10 PRBs for URLLC, eMBB, and mMTC. Aggregate demand therefore exceeds the budget by about five per cent, while URLLC and eMBB demand stays below their own caps of 33 and 66 PRBs. Whenever the slices' admitted demand exceeds B, they share B in proportion to that demand, so every admitted mMTC request takes resources from URLLC and eMBB. Unserved demand carries forward as backlog within an episode, which drives the SLA margin of Eq. (2). Every episode starts with empty queues, and per-gNB load multipliers drawn from $[0.9, 1.0]$ keep the cluster heterogeneous. This shared-budget design replaces an earlier simulator that served each slice independently at about a quarter of the budget, where accepting every request was harmless. Section 4.4 shows how a trivial accept-all baseline exposed that, and verifies that shedding mMTC is both the reward-optimal and the SLA-protecting behaviour in the current design. We also use a second, harder environment, Env B, which keeps Env A and adds two mechanisms. First, each gNB's mean demand follows $0.95\,(1 + 0.3\sin(2\pi(s + \phi_i)/60))$ times its nominal shares, where s is the step within

the episode and the phase ϕ_i is redrawn for every gNB at every episode, so demand swings between about 0.67 and 1.24 times the budget. Second, a gNB's usable budget shrinks with its neighbours' load on the previous step, $B_i(t) = B\,(1-0.15\,\bar{u}_{\mathcal{N}(i)}(t-1))$, where $\bar{u}_{\mathcal{N}(i)}$ is the mean PRB utilisation of i's neighbours in the same adjacency graph the encoder reads. This is a first-order model of inter-cell interference, which a gNB observes only through its own queues. The reward weights are unchanged from Env A. In Env B shedding every mMTC request is no longer optimal, since it leaves URLLC compliance at 62.5% at three gNBs, whereas a rule that also sheds eMBB when the URLLC queue builds keeps it at 98.9%. Both constants were calibrated on pilot seeds with rules only, before any reported seed was run. Per-slice compliance counts a slice as compliant only when every gNB meets it, so in Env B it saturates at nineteen gNBs. Env B is therefore run at three and seven gNBs, and the nineteen-gNB evidence comes from Env A.

The concrete rig behind every live measurement is the single-host, software-radio, O-RAN stack of Fig. 3, which runs without a near-real-time RAN Intelligent Controller (RIC). We omit the RIC because the ORANSlice fork applies slicing control directly through a lightweight UDP E2 agent, which exercises the real per-slice PRB-ratio primitive without the overhead of a separate RIC on a single commodity host.

4 The AXIOM Architecture

AXIOM differs from its two DQN baselines in exactly what each policy sees and shares. AXIOM runs one graph-attention encoder over the whole gNB cluster, trains that encoder jointly from every agent's experience, and reads out one dedicated Q-head per slice, so every gNB decides from a shared view of the cluster that the adjacency graph shapes. The single-agent DQN keeps one controller over a flattened joint state, with no per-gNB structure and no graph. The independent DQN gives each gNB its own learner that sees only local features and never shares parameters or reads the cluster topology. The two baselines are therefore ablations of AXIOM that remove the shared encoder and the graph input together, so any gap between them and AXIOM measures what those components add jointly. A third variant that keeps everything except attention isolates the graph's own contribution in Section 5.2.

4.1 Shared Encoder, Decentralised Q-Heads

Fig. 4a shows the architecture. At every control step, each gNB forms its own per-slice feature row x_i. The three features are PRB-used-ratio, congestion-level, and queue-length-norm. These rows stack into a joint node-feature matrix $X \in \mathbb{R}^{N\times d}$ over the N gNB nodes, with d features each. A two-layer, H=4-head GAT [29] produces one embedding per gNB. Each head h decomposes additive attention rather than scoring a concatenation. It uses a learned source vector a_s^h and destination vector a_d^h, applied to the shared linear projection W^h. Here ϕ is the leaky rectifier of slope 0.2, and $\mathcal{N}_i$ is the set of neighbours of node i under the adjacency mask,

$$\varepsilon_{ij}^h = \phi\big(a_s^{h\top} W^h x_i + a_d^{h\top} W^h x_j\big), \qquad \alpha_{ij}^h = \frac{\exp \varepsilon_{ij}^h}{\sum_{l\in\mathcal{N}_i} \exp \varepsilon_{il}^h}. \tag{3}$$

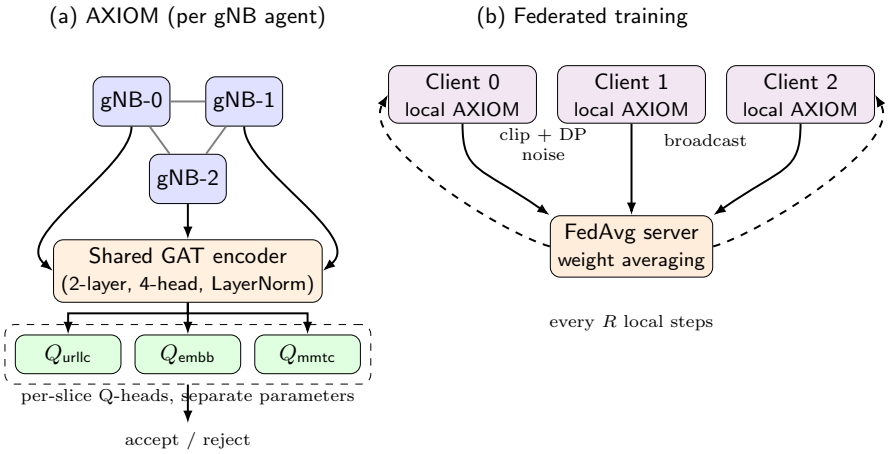


Fig. 4 AXIOM architecture. a, A shared graph-attention encoder over the fully-connected 3-gNB topology produces one embedding per node, consumed at execution time by per-slice Q-heads using only that node's row. b, The federated variant, each gNB a client synchronised by periodic FedAvg with an optional DP-SGD clip-and-noise step.

Here ε_{ij}^{h} is the unnormalised attention score from node i to neighbour j in head h, and α_{ij}^{h} is its normalisation over i's neighbours, with l ranging over $\mathcal{N}_i$. Hidden layers concatenate the H heads under an ELU nonlinearity, while the final layer averages them. A per-layer normalisation operator $\mathrm{LN}(\cdot)$ then rescales each node's embedding to zero mean and unit variance, and produces the final per-gNB embedding z_i,

$$z_i = \mathrm{LN}\Big(\tfrac{1}{H}\textstyle\sum_{h=1}^{H}\sum_{j\in\mathcal{N}_i}\alpha_{ij}^{h}\,W^{h}x_j\Big). \tag{4}$$

Centralisation comes from this encoder, whose parameters every agent's loss updates jointly, so it sees the full cluster state during training. Decentralised execution comes next, since at action-selection time each agent consumes only its own embedding row z_i to decide its own pending requests.

The `LayerNorm` and the per-slice heads are deliberate design choices. With a single shared Q-head, all three slices update the same parameters, and URLLC's large priority weight produces gradients that swamp mMTC's much smaller signal, a standard shared-parameter class imbalance. One small multi-layer perceptron (MLP) per slice, selected by the context one-hot, gives each slice its own parameters, and the `LayerNorm` stops an embedding from encoding congestion through its magnitude alone. That normalisation also hides the absolute queue and congestion levels on which a state-dependent admission rule depends, so each node's raw features are concatenated onto its normalised embedding before the Q-heads, $Q_{\phi_k}([z_i, x_i])$, and below we write z_i for that concatenation. This skip connection, together with a learning rate of 10^{-3} in place of the earlier 10^{-4}, is the only change from the original AXIOM design. Both were chosen on pilot seeds of the time-varying environment of Section 3, where the earlier AXIOM learned only the static shedding rule, and both were fixed before any

Algorithm 1 AXIOM: centralised training, decentralised execution

Require: encoder f_θ, per-slice Q-heads $\{Q_{\phi_k}\}$, targets $f_{\theta^-}, \{Q_{\phi_k^-}\}$, buffer $\mathcal{D}$, adjacency A, discount γ, sync period C

for each step t, observing joint node features X_t **do**
  $E_t \leftarrow f_\theta(X_t, A)$ {centralised, one row per gNB}
  for each pending request at gNB i, slice k, pick $a \leftarrow \arg\max_a Q_{\phi_k}([E_t[i], X_t[i]])[a]$, ϵ-greedy {decentralised, own row only}
  step env, store $(X_t, X_{t+1}, \{(i,k,a)\}, r_t, \text{done})$ in $\mathcal{D}$
  if $|\mathcal{D}| \geq$ warmup **then**
    sample minibatch B; $E_B \leftarrow f_\theta(X_B, A)$, $E'_B \leftarrow f_{\theta^-}(X'_B, A)$
    per request, $y \leftarrow r + \gamma(1-\text{done})\max_{a'} Q_{\phi_k^-}(E'_B[i])[a']$
    $\mathcal{L} \leftarrow$ mean per-request Huber loss, never summed per step; update $\theta, \{\phi_k\}$, clip grad-norm 1.0
  end if
  every C episodes, $\theta^- \leftarrow \theta$, $\phi_k^- \leftarrow \phi_k$
end for

reported seed was run. A seed here is one independent random initialisation of network weights and environment stream, and we report every seed rather than a single best run.

Algorithm 1 states the training loop, where the encoder updates from every request in a sampled minibatch regardless of gNB or slice. This is centralised training, while action selection at rollout time consults only the acting agent's own embedding row, which is decentralised execution. Every pending request at gNB i, slice k is its own temporal-difference sample, and we never sum requests across a step before the loss. For a request with immediate reward r, next-state embedding z'_i, and candidate next action a', the bootstrapped target y at discount γ and the minibatch loss are

$$y = r + \gamma\,(1-d)\max_{a'} Q_{\phi_k^-}\big(z'_i\big)[a'], \qquad \mathcal{L} = \frac{1}{|B|}\sum_{(i,k,a)\in B} \ell_\delta\Big(Q_{\phi_k}(z_i)[a] - y\Big), \tag{5}$$

where Q_{ϕ_k} is slice k's online Q-head, $Q_{\phi_k^-}$ its target copy, d the episode-done flag, and B the sampled minibatch of requests. We use the Huber loss ℓ_δ in place of squared error, which keeps the deeper encoder stable under long bootstraps. An earlier variant passed each gNB's summed per-request values through a QMIX-style monotonic mixer [22]. Because the number of pending requests changes from step to step, that mixed target changed scale with the request count, and the mean training loss diverged, rising from about 76 over the first fifth of training steps to about 409 over the last fifth. Treating each request as its own sample removes that dependence.

4.2 Federated Training with Differential Privacy

Fig. 4b shows the federated variant. In this variant, each gNB becomes a federated-learning client. It holds a private copy of the same network and trains only on its own requests. Transitions are partitioned by owning agent, so no client sees another's

raw admission data. Each client's GAT encoder still consumes the full joint node-feature snapshot. Every R local steps, the server averages all clients' online weights by FedAvg [17] and broadcasts the result back. We also implement FedProx [16], which adds a proximal term that pulls local weights toward the last broadcast global model. We sweep it at $\mu \in \{0.01, 0.1, 1.0\}$ on three seeds. FedProx gives no consistent improvement over FedAvg, as Section 5.4 reports, so we keep FedAvg.

Differential privacy (DP) bounds how much any one client's training data can change what the shared model reveals. We apply it as a differentially private stochastic gradient descent (DP-SGD) step [1]. Each client clips its minibatch gradient to a fixed norm C_{dp} and, when privacy is enabled, adds Gaussian noise of standard deviation σC_{dp} before its optimiser step. The clip applies to the whole minibatch gradient rather than to each example, so changing any of the client's data in one step moves the clipped gradient by at most $2C_{\mathrm{dp}}$, and the guarantee covers the client's whole step batch. We account for it with zero-concentrated differential privacy (zCDP) [6], where ε and δ are the standard privacy-loss and failure-probability parameters. Composing the Gaussian mechanism over the m noisy steps a client takes gives

$$\rho = \frac{2m}{\sigma^2}, \qquad \varepsilon(\delta) = \rho + 2\sqrt{\rho \ln(1/\delta)}. \tag{6}$$

We credit no amplification by subsampling, because minibatches come from a growing replay buffer that reuses records, so Eq. (6) is an upper bound. The clip is applied whether or not noise is added, so a $\sigma{=}0$ run and a $\sigma{>}0$ run differ only in the injected noise. The noise protects each client's own admission decisions and rewards, but not its congestion features, which every client's encoder reads from the shared node-feature snapshot by design. Each federated round therefore runs R local Huber-TD steps per client with an optional FedProx proximal term $\frac{\mu}{2}\|\theta_i - \bar{\theta}\|^2$, clips and optionally perturbs the gradient, and closes with a FedAvg average that is broadcast back.

4.3 Experimental Setup

Every campaign uses 300 training and 50 evaluation episodes per seed, and each seed is evaluated on a held-out environment stream. The three-gNB campaigns use 30 seeds, 900 to 929, in each environment, with a disjoint replication sample of seeds 1000 to 1009. The scaling campaigns use 15 seeds per cell at seven gNBs and 30 per cell at nineteen. We compare AXIOM with three learned baselines and several rules. The single-agent DQN is one controller over the flattened joint state. The independent DQN gives each gNB its own DQN with no parameter sharing and no topology information. Both DQN policies are ablations, deliberately reduced variants with a component removed so that its effect can be isolated. MAPPO [31] is a published multi-agent baseline with a shared actor over each gNB's own features and a centralised critic over the joint state. It trains on-policy with proximal policy optimisation, for the same 300 episodes and on the same global reward. Its learning rate, epochs, and minibatch size were tuned on pilot seeds only, because untuned it collapsed into accepting every request in four of five pilot seeds.

Rules need no learning, and accept-all simply admits every request. The oracle rule of Env A sheds mMTC whenever its gNB's congestion level reaches a threshold τ, chosen on training streams only, and the selected value is τ=0, so the rule sheds every mMTC request. In Env B we add two tuned rule families, each chosen on training streams over a grid of thresholds and evaluated on the held-out streams. The local family sheds mMTC on the gNB's own congestion or priority-slice queue and sheds eMBB on its URLLC queue, while the neighbour-aware family also reads the mean congestion of the gNB's neighbours. Two further arms isolate AXIOM's components, namely the DQN baselines at the lower learning rate of 10^{-4}, and an AXIOM variant whose graph encoder is replaced by a per-node MLP of the same width that ignores the adjacency.

All learned arms share the discount factor $\gamma = 0.95$ and the training budget. The value-based arms also share the Adam optimiser, an epsilon schedule decaying from 1.0 to 0.05 at a per-episode factor of 0.985, and target synchronisation every 10 episodes. AXIOM and the DQN baselines train at learning rate 10^{-3}. The GAT encoder uses hidden and output dimension 16, feeding per-slice Q-heads of hidden dimension 32. A sensitivity study repeats the learned arms and the oracle rule in Env A on 10 seeds under three changed conditions. Two change the load, to 1.02 and 1.07 times the nominal shares, and the third lowers eMBB's priority weight from 3.5 to 2.5 and raises mMTC's from 0.3 to 0.6. A second study repeats AXIOM in Env B on 10 seeds with learning rates of 3×10^{-4} and 3×10^{-3} and with an encoder twice as wide.

The federated campaign runs in Env A on seeds 900 to 909, with a five-level noise sweep $\sigma \in \{0, 0.5, 1, 2, 4\}$ and a replication on seeds 1000 to 1009. Each federated round runs 50 local steps under FedAvg with a gradient-clip norm of 1.0. All confidence intervals (CIs) are 95% bootstrap percentile intervals over 10,000 resamples. Paired comparisons use the Wilcoxon signed-rank test, and each family of comparisons against AXIOM is Holm-corrected for multiple testing.

The settings, seeds, and hypotheses of the final campaign were fixed in a dated pre-registration, committed before any reported seed was run. It includes the change to AXIOM's learning rate and skip connection and the constants of Env B, both chosen on pilot seeds 2000 to 2009. It states four hypotheses, all fixed before any result. H1 is that AXIOM beats both trivial rules in Env B. H2 covers how AXIOM compares with every learned baseline and tuned rule in Env B. H3 asks whether graph attention beats the per-node encoder in Env B at seven gNBs. H4 covers how AXIOM compares with every baseline at nineteen gNBs in Env A, over 30 seeds per layout.

A validation screen checks every trained checkpoint of every learned arm before evaluation. Each checkpoint plays five greedy episodes on its own training stream, never on the held-out stream, and fails the screen if it blocks no request or if fewer than 90% of its blocks target mMTC. We report every seed whether or not it passes, and separately report a two-seed deployment protocol that trains a pair of seeds and deploys the first one to pass. The same screen applies to every learned arm, so it gives AXIOM no advantage the baselines lack. In Env B the reward-optimal policy sheds eMBB as well as mMTC, so the 90% criterion would fail good policies there. A pre-registration amendment, recorded before any result was read, therefore makes the no-blocking criterion the primary screen in Env B.

4.4 Evaluation Safeguards

Three safeguards make the results below trustworthy, and Table 1 lists the failure each one caught during this work. A standard pipeline would have reported every one of them as a clean result.

The first is a trivial-policy check, in which we evaluate the simplest rules the environment admits before trusting any learned policy, namely accepting every request and shedding one slice outright. In the first version of our simulator, every slice was served independently against its own ceiling at about a quarter of the gNB's budget. There, accepting every request earned 14.45 reward per step and kept URLLC and eMBB at 100% SLA compliance, more than any learned policy, so admission control had nothing to do and policies that appeared to fail by never blocking were in fact optimal. The shared-budget simulator of Section 3 removes that loophole. Over the same 30 held-out seeds, accepting every request now earns 9.89 reward per step and lets eMBB compliance fall to 56.8%, whereas shedding every mMTC request earns 12.84 and keeps URLLC and eMBB at 100%. Every result in this paper comes from the shared-budget simulator.

The second is a correctness-aware set of metrics, computed from data every run already logs. Mean reward per step is the training objective. Block precision is the fraction of blocked requests that correctly target mMTC, the slice the reward calibration makes reject-optimal. Writing b_{mMTC} and b for the mMTC-targeted and total blocks over an evaluation, it is $\pi = b_{\mathrm{mMTC}}/b$ for $b > 0$ and undefined for $b = 0$, so a seed that never blocks is counted as a collapse rather than scored as a success. Per-slice SLA compliance then shows whether the shedding actually protects URLLC and eMBB. We do not use the common all-slice compliance metric, because correct shedding gives up mMTC's own SLA by design, so that metric scores a correct policy and a failed one alike.

The third is a reproduction-versus-replication discipline, in which we recompute every headline comparison on an independent, disjoint seed sample and report both. Reproduction re-runs the same seeds, whereas replication draws fresh ones, so only replication can catch a result that is an artifact of one seed sample. It caught one such artifact in this work. A weaker replication led us to inspect step-level traces, which showed that queue backlog carried over between episodes. With only about half a PRB of headroom once mMTC is shed, a backlog built by one early transient never drained, so a single bad episode spoiled every later one for every policy, the oracle rule included. Every result in this paper comes from the corrected simulator, which empties all queues at the start of each episode.

5 Results

We compare AXIOM with three learned baselines and a family of rules in two environments. AXIOM is the architecture of this paper, a shared encoder that reads the whole base-station cluster and feeds one small decision head per slice. The single-agent DQN flattens the cluster into one controller that sees no per-gNB structure. The independent DQN gives each gNB its own learner that never looks at its neighbours. MAPPO shares one actor across gNBs and trains it with a centralised critic. The rules range

Table 1 Evaluation failures this paper's safeguards caught during the work, none of which a standard pipeline would have surfaced.

Failure	Missed by	Caught by	Outcome
Inert simulator (§3)	Learned-policy metrics	Accept-all baseline	Shared-budget rerun
No-op scored as fine	All-slice compliance	Precision, per-slice SLA	Collapse made visible
Repro. gap (§5.5)	Single-sample claim	Repro-vs-replication	Churn claim retracted
Backlog carry-over (§4.4)	Per-seed means	Replication, step traces	Episode reset, rerun
Static AXIOM in Env B (§4.1)	Env A results	Env B pilot seeds	Faster learning rate, skip
Screen miscalibrated (§4.3)	Env A screen design	Pilot screen check	Pre-registered amendment
Scheduler floor (§5.9)	Cross-layer floor	Direct instrumentation	100k+ readings clear

from accept-all and static mMTC shedding, the optimum of Env A, to threshold rules tuned on training streams.

The comparison rests on four quantities, the first being mean reward per step, the objective the policies train on, so higher is better. Block precision is the share of a policy's rejections that land on mMTC. Per-slice SLA compliance shows whether a policy's shedding actually protects URLLC and eMBB, and we summarise it as priority-slice compliance, the mean of the two. Collapse names a failure mode in which a trained policy stops discriminating and simply accepts every request. Table 2 sets out how the four learned policies stand, and the subsections that follow give the evidence.

The headline is that AXIOM is the most reliable learned policy as the cluster grows in Env A, where it keeps priority-slice compliance near 97% at seven and nineteen gNBs while every baseline falls to between 56% and 76%. It is not, however, uniformly best across environments. In Env B a tuned threshold rule beats every learned policy, and at seven gNBs AXIOM does no better than the DQN baselines or the static rule.

Table 2 How the four learned policies compare. Env A has static load, Env B time-varying load with inter-cell coupling. Priority SLA is the mean of URLLC and eMBB compliance.

Property	AXIOM	Single-agent DQN	Independent DQN	MAPPO
Design	Shared encoder, per-slice heads	One flattened controller	One learner per gNB	Shared actor, central critic
Parameters at execution	7,782 at any size	21,250 to 41,730	55,302 to 350,246	5,122
Env A reward, 3 gNBs	12.70, 98.9% of oracle	12.39	11.59	10.01
Env A priority SLA, 7 and 19 gNBs	97.1% and 97.3%	76.2% and 65.3%	74.6% and 56.4%	67.7% and 60.6%
Collapse, Env A, 19 gNBs	1/90	84/90	3/90	0/90
Env B reward, 3 gNBs	8.83	7.49	7.79	6.97
Env B priority SLA, 7 gNBs	35.1%	44.3%	39.9%	35.9%
Federated, noise-free	No significant cost	Not applicable	Not applicable	Not applicable

Table 3 Env A, three gNBs, 30 seeds per arm, 95% bootstrap CIs. Precision averages the seeds that block at all. SLA columns give per-slice compliance.

Arm	Reward/step	Precision	Collapsed	URLLC	eMBB
Oracle rule	12.835 [12.77, 12.91]	1.000	0/30	1.000	1.000
AXIOM	12.698 [12.49, 12.86]	0.992	0/30	0.998	0.979
AXIOM, MLP encoder	12.640 [12.46, 12.79]	0.986	0/30	1.000	0.976
Single-agent DQN	12.387 [12.11, 12.63]	0.990	0/30	0.994	0.941
at LR 10^{-4}	11.477 [10.61, 12.09]	0.944	0/30	0.979	0.866
Independent DQN	11.587 [11.11, 12.02]	0.962	0/30	0.993	0.865
at LR 10^{-4}	11.119 [10.73, 11.50]	0.878	0/30	0.985	0.794
MAPPO	10.013 [9.30, 10.73]	0.707	2/30	0.978	0.576

5.1 Static Load, Three gNBs

Table 3 and Fig. 5 report the thirty-seed campaign in Env A. AXIOM never collapses, 99.2% of its blocks land on mMTC, and it keeps URLLC and eMBB at 99.8% and 97.9% SLA compliance. Its mean reward of 12.698 per step is 98.9% of the oracle rule's 12.835, and the two tie exactly in 22 of the 30 seeds. AXIOM leads the independent DQN by +1.110 per step, with 95% CI [0.60, 1.65], and MAPPO by +2.685, with 95% CI [2.01, 3.35], both at Holm-adjusted $p \leq 0.0002$. Its lead over the single-agent DQN, +0.311 with 95% CI [−0.01, 0.64], does not survive correction on thirty seeds, at $p = 0.066$. MAPPO trails both DQN baselines here, because it sheds eMBB as well as mMTC, which lets eMBB compliance fall to 57.6%.

A disjoint replication on seeds 1000 to 1009 is stronger for AXIOM. There it leads the single-agent DQN, the independent DQN, and MAPPO by +1.083, +0.799, and +3.806 per step at Holm-adjusted $p \leq 0.023$, while sitting only 0.018 below the oracle. Pooled over all 40 seeds, AXIOM leads the three baselines by +0.504, +1.032, and +2.965 at Holm-adjusted $p \leq 0.002$, and falls 0.108 short of the oracle, a small gap

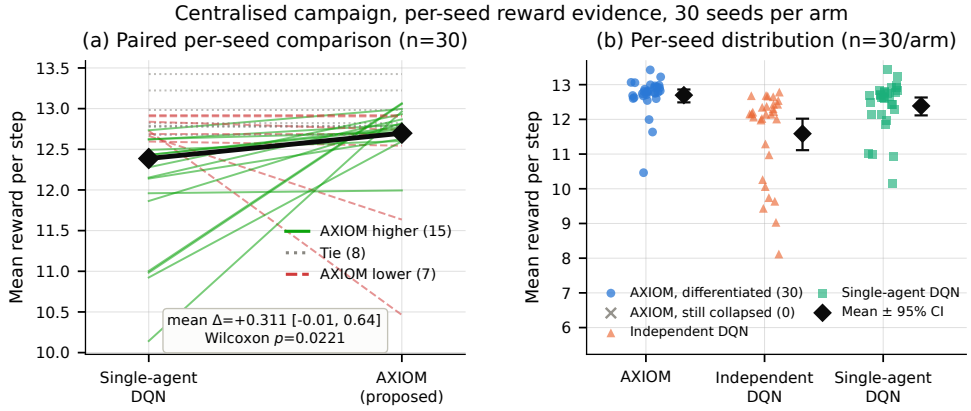


Fig. 5 Env A, three gNBs, per-seed evidence. a, Paired per-seed mean reward per step of AXIOM versus single-agent DQN, coloured by win, tie, or loss. b, Per-seed reward distribution for AXIOM and the two DQN baselines.

that is significant at $p = 0.034$. Fresh seeds in Section 5.8 do not reproduce the lead over the single-agent DQN, however. The original AXIOM, at learning rate 10^{-4}, came slightly closer to the oracle in this environment, tying it in 39 of 40 seeds. The change to the current design therefore trades a little static-load accuracy for the ability to adapt that Section 5.3 shows.

5.2 Ablations and Sensitivity

Three checks test where AXIOM's behaviour comes from. First, the learning rate matters, for AXIOM and its baselines alike. Every learned arm now trains at 10^{-3}, and the DQN baselines lose reward when rerun at 10^{-4}, falling to 11.477 and 11.119 per step. AXIOM depends on its rate even more in Env B. There the original AXIOM, at 10^{-4}, earns only 6.496 per step on seeds 900 to 909, 2.37 below the current AXIOM in all ten seeds, and a rate of $3{\times}10^{-4}$ reaches 7.220. A rate of $3{\times}10^{-3}$ and an encoder twice as wide both stay within 0.19 of the current AXIOM, with no significant difference.

Second, graph attention adds nothing measurable anywhere we tested it. In Env A the per-node MLP variant ties AXIOM at three gNBs, with a difference of +0.058 and 95% CI [−0.17, 0.26], and at nineteen gNBs on the ring layout, +0.31 per step with 7 of 12 seeds tied. Env B's inter-cell coupling gives neighbour information real value, yet the pre-registered test H3 at seven gNBs even leans towards the MLP. The MLP keeps 1.12 and 0.95 more reward per step on the ring and hex layouts, winning 10 of 15 seeds in each, although neither difference survives Holm correction, at $p = 0.061$ and $p = 0.095$. AXIOM's gains therefore come from its shared encoder, per-slice heads, and joint training rather than from attention. One explanation is that a gNB's own queues already reflect most of what its neighbours do to it, so attending to them adds little. Following the pre-registration, the title of this paper therefore no longer names graph attention.

Third, AXIOM's behaviour holds in Env A when the environment changes. Under the three conditions of Section 4.3, namely load at 1.02 and 1.07 times the nominal shares and the changed slice weights, AXIOM stays within 0.13 of the oracle rule's reward with no significant difference. It keeps a positive lead over both DQN baselines under every condition, but at ten seeds only two of those six leads survive correction. They are its lead over the independent DQN at 1.07 times the load, +1.073 at $p = 0.006$, and over the single-agent DQN under the changed weights, +1.264 at $p = 0.041$.

5.3 Time-Varying Load with Inter-Cell Coupling

Env B tests the case a static benchmark cannot, where no fixed rule is optimal. Table 4 and Fig. 6 report the thirty-seed campaign at three gNBs and fifteen seeds per layout at seven. First, adapting to the load pays off clearly. Shedding every mMTC request, the optimum of Env A, now leaves URLLC compliance at 62.5%. AXIOM beats that rule by +2.495 reward per step and accept-all by +4.476, winning all 30 seeds against each at Holm-adjusted $p < 0.0001$, as hypothesis H1 predicted. AXIOM keeps URLLC at 95.8% by shedding eMBB as well as mMTC when URLLC queues build.

Second, among learned policies AXIOM leads at three gNBs. It beats the independent DQN, the single-agent DQN, and MAPPO by +1.038, +1.337, and +1.863

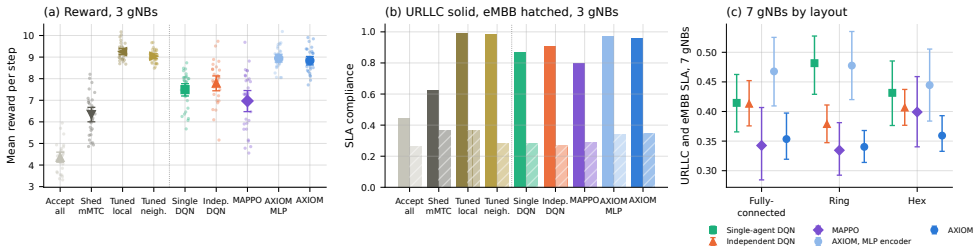


Fig. 6 Env B. a, Reward at three gNBs, 30 seeds, per-seed points behind the mean and 95% CI. b, URLLC and eMBB compliance at three gNBs. c, Priority-slice compliance of the learned arms at seven gNBs by layout, 15 seeds per cell.

reward per step and also on priority-slice compliance, winning 24 to 29 of 30 seeds, all at Holm-adjusted $p < 0.0001$. The disjoint ten-seed replication points the same way, with AXIOM winning 7 or 8 of 10 seeds against each, but no lead survives correction there.

Third, a tuned rule beats every learned policy. The best local rule sheds every mMTC request and sheds eMBB whenever the URLLC queue passes 0.2, and it earns 9.252 per step, 0.420 more than AXIOM, winning 22 of 30 seeds at $p < 0.0001$. The neighbour-aware rule earns slightly less, at 9.041, so neighbour information did not help the rules either. The rule encodes domain knowledge of which slice to shed under which pressure, which AXIOM must learn from the reward alone in 300 episodes.

At seven gNBs every learned policy falls further behind. The tuned rules earn 26.5 to 27.3 per step against 20.3 to 24.6 for the learned policies, and they beat AXIOM in all 15 seeds under every layout. AXIOM itself earns 23.4 to 23.6, no more than the static rule's 23.6, and keeps priority-slice compliance at 34% to 36%, close to the static rule. On the ring layout that is below the single-agent DQN, at Holm-adjusted $p = 0.011$. AXIOM's only significant leads over a learned baseline at this size are over MAPPO's reward, under the fully-connected and ring layouts. In Env B, learned multi-agent admission control therefore beats a static rule only at three gNBs, and never beats a rule tuned with domain knowledge.

Table 4 Env B, three gNBs, 30 seeds per arm, 95% bootstrap CIs. Rules above the line, learned policies below it.

Arm	Reward/step	URLLC	eMBB	mMTC
Accept all	4.356 [4.13, 4.60]	0.446	0.265	0.955
Shed all mMTC	6.337 [6.00, 6.67]	0.625	0.367	0.371
Tuned local rule	9.252 [9.12, 9.40]	0.989	0.367	0.390
Tuned neighbour-aware rule	9.041 [8.93, 9.15]	0.988	0.283	0.901
AXIOM	8.832 [8.64, 9.03]	0.958	0.344	0.406
AXIOM, MLP encoder	8.948 [8.77, 9.13]	0.974	0.337	0.452
Single-agent DQN	7.495 [7.20, 7.77]	0.866	0.284	0.412
Independent DQN	7.794 [7.44, 8.13]	0.910	0.270	0.441
MAPPO	6.968 [6.47, 7.45]	0.797	0.289	0.579

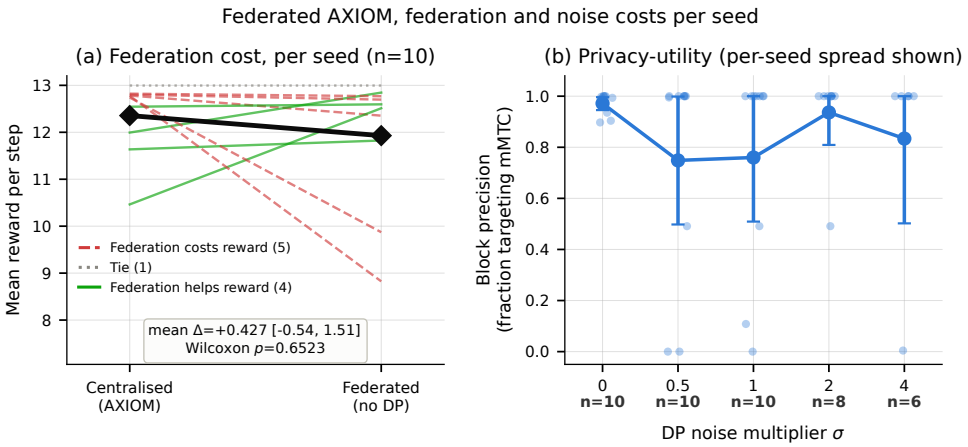


Fig. 7 Federated campaign in Env A, 10 seeds. a, Paired per-seed federation cost on mean reward per step, not significant at $p = 0.65$. b, Block precision versus DP noise multiplier σ, over the seeds that block.

5.4 Federated Training and DP

With federation switched on and no noise, mean reward per step on the ten shared seeds is 12.356 for centralised AXIOM and 11.929 for the federated variant. That is a difference of +0.427 with 95% CI [−0.54, 1.51] and $p = 0.65$, so federation shows no significant cost, although it is more variable, with five seeds losing and four gaining. The disjoint replication shows a larger cost of +0.968, with 95% CI [0.18, 1.96], which falls just short of significance at $p = 0.055$. FedProx on three seeds gives no consistent picture, with μ=0.01 above FedAvg and μ of 0.1 and 1.0 below it. The DQN baselines are not federated, because averaging the independent learners' weights would turn them into a parameter-sharing learner that is no longer the ablation, and the single-agent DQN is one central controller with nothing to federate.

Adding DP-SGD noise to every step lowers reward by 0.77 to 1.25 per step against centralised AXIOM at every tested noise multiplier, from σ=0.5 to σ=4. It also lowers eMBB compliance to between 74.3% and 83.0%, and two and four of the ten seeds collapse at σ of 2 and 4. None of these drops survives Holm correction at ten seeds, as Fig. 7b reflects. The formal guarantee is weak at these settings. Each client takes 17,969 noisy steps, so Eq. (6) gives ε of about 146,000, 37,000, 9,600, and 2,600 at δ=10^{-5} for σ of 0.5, 1, 2, and 4, far from the single-digit values that carry meaning. Per-step DP-SGD therefore delivers no meaningful guarantee here. A useful guarantee would need noise on the roughly 360 per-round model updates rather than on every step, which Section 6 lists as future work.

5.5 Disruption

We test every policy's frozen Env A checkpoint against three mid-episode disruptions, injected at evaluation time with no retraining. A 'gNB dropout' forces one gNB to

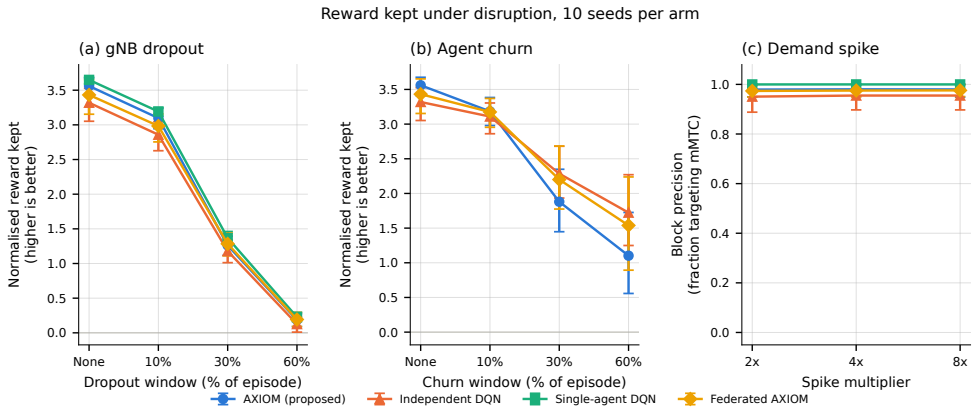


Fig. 8 Disruption campaign, 10 seeds per arm, 95% CIs. a, b, Normalised reward kept under dropout and churn, with the undisrupted level at None. c, Block precision under demand spikes.

reject everything and blanks its row in the shared feature matrix for a window. A 'demand spike' briefly multiplies the arrival rate from outside the policy. An 'agent churn' swaps one gNB's decisions for a fresh, untrained copy of its own policy for a window and leaves the others intact. Churn applies to every genuinely multi-agent policy but not to the single-agent one. For dropout and churn the severity is how long the window lasts, at 10%, 30%, or 60% of the episode. For the spike it is the arrival multiplier, at 2×, 4×, or 8×. Because raw reward grows mechanically with request volume, we report a volume-normalised reward over an episode of T steps, $\bar{r} = \frac{1}{T}\sum_{t=1}^{T} r_t / \max(n_t, 1)$, which divides each step reward r_t by its own pending-request count n_t before averaging.

Fig. 8 plots the reward each policy keeps. Undisrupted, AXIOM earns 3.56 normalised reward, the single-agent DQN 3.65, and the independent DQN 3.32. Dropout and churn hurt every policy significantly at every severity, in 9 or 10 of 10 seeds at Holm-adjusted $p \leq 0.041$. AXIOM's cost grows from 0.46 to 3.37 across the dropout windows and from 0.38 to 2.46 across the churn windows. A dropped gNB rejects every request, including URLLC and eMBB, so its penalties dominate the episode whatever the policy does. No gap between AXIOM and any baseline survives Holm correction under any disruption. Under the two longer churn windows AXIOM keeps less reward than the independent DQN, 1.88 and 1.10 against 2.29 and 1.72, though not significantly. Demand spikes leave AXIOM's decisions largely intact, with block precision at 0.98, against 1.00 for the single-agent DQN and about 0.95 for the independent DQN. A disjoint replication on seeds 1000 to 1009 reproduces this pattern, with every dropout and churn cost significant at $p \leq 0.035$ and no significant gap between policies.

Replication first proved its worth in this campaign, on the earliest simulator. There, a ten-seed sample suggested that the independent DQN was immune to churn, since it never reads another agent's features, and a disjoint replication overturned that reading with a significant cost at every severity. In the shared-budget environment the primary sample already shows that churn hurts every policy, so the corrected reading holds.

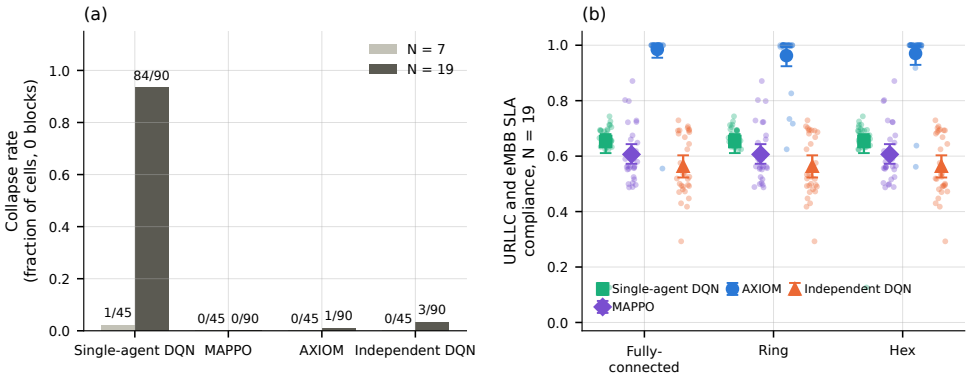


Fig. 9 Env A scaling. a, Collapse rate pooled over layouts, with collapsed over total cells. b, Mean URLLC and eMBB compliance at 19 gNBs by layout, 30 seeds per cell, per-seed points behind the mean and 95% CI.

5.6 Cluster-Size Scaling

We grow the cluster in Env A from three gNBs to seven and nineteen under three layouts, fully-connected, ring, and hex-grid. Every gNB keeps its own shared 100-PRB budget and its own demand, so per-gNB load stays the same as the cluster grows. Each cell runs 15 seeds at seven gNBs and 30 at nineteen, where the earlier comparison had been underpowered. The independent DQN ignores the layout, so it gives identical results under all three layouts, which checks that the layout changes nothing else. The single-agent DQN also ignores the layout, yet its results vary between layouts in some seeds, because its training is not bit-reproducible. We test the per-layout gaps on reward and on priority-slice compliance, Holm-correcting the 18 comparisons at each size.

At seven gNBs AXIOM keeps URLLC and eMBB at 99.8% and 94.4%, against 91.4% and 61.0% for the single-agent DQN, 94.2% and 55.0% for the independent DQN, and 87.0% and 48.4% for MAPPO. Under every layout it leads all three baselines on priority-slice compliance by 17.9 to 29.9 percentage points and on cluster reward by 3.40 to 7.07 per step. All 18 comparisons hold after Holm correction, at adjusted $p \leq 0.011$.

At nineteen gNBs the gaps widen, and they answer hypothesis H4. AXIOM keeps URLLC and eMBB at 99.4% and 95.1%, a pooled priority-slice compliance of 97.3% with 95% CI [95.2%, 99.0%]. The single-agent DQN, the independent DQN, and MAPPO keep 65.3%, 56.4%, and 60.6%. Under every layout AXIOM leads every baseline by 31.0 to 42.1 percentage points of compliance and by 10.7 to 18.6 reward per step, winning 28 to 30 of 30 seeds. All 18 comparisons hold after Holm correction, at adjusted $p \leq 0.0001$. Fig. 9a shows the single-agent DQN collapsing into accepting every request in 84 of its 90 cells, against 3 for the independent DQN, 1 for AXIOM, and none for MAPPO. The original AXIOM kept priority-slice compliance at only 79.1% at this size, so the faster learning rate matters most at scale.

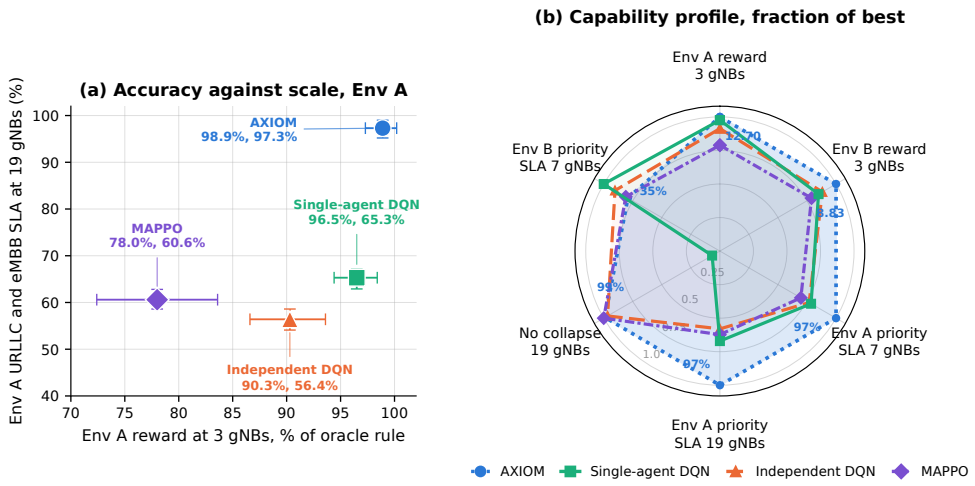


Fig. 10 AXIOM against the three learned baselines. a, Env A reward at three gNBs as a share of the oracle rule against compliance at 19 gNBs, with 95% CIs. b, Six measured axes, each as a fraction of the best learned policy on it.

The pre-registered validation screen rarely rejects AXIOM in Env A. It fails 1 of 30 seeds at three gNBs, 1 of 45 at seven, and 5 of 90 at nineteen, including the one seed that collapses. The screen fails 89 of the single-agent DQN's 90 seeds at nineteen gNBs. It also fails many MAPPO seeds, because MAPPO also blocks eMBB, so the 90% criterion flags it. In Env B no learned policy collapsed at any size.

Fig. 10a places the four learned policies on one map of accuracy at three gNBs against priority-slice protection at nineteen, in Env A. AXIOM sits highest on both axes, at 98.9% of the oracle reward and 97.3% compliance. The single-agent DQN reaches 96.5% and 65.3%, the independent DQN 90.3% and 56.4%, and MAPPO 78.0% and 60.6%. The radar in Fig. 10b adds Env B reward at three gNBs, collapse resistance, and Env B compliance at seven gNBs. AXIOM leads on five of the six axes shown. The exception is Env B compliance at seven gNBs, where the single-agent DQN leads.

5.7 Training Dynamics and Complexity

Fig. 11 shows how the four learned policies train. MAPPO rises fastest in the first few episodes but plateaus lowest in every cell, ending at 6.4, 16.0, and 62.9 in the three panels. The value-based arms rise more slowly but further. At nineteen gNBs AXIOM keeps improving to about 85 per step over the last 20 training episodes, while the two DQN baselines settle near 71 and 74. Table 5 sets out the cost. AXIOM's shared encoder keeps its parameter count at 7,782 at every cluster size, whereas the independent DQN grows from 55,302 to 350,246 parameters and the single-agent DQN's input grows with the cluster. AXIOM's greedy decisions take 1.0 to 4.8 ms per control step

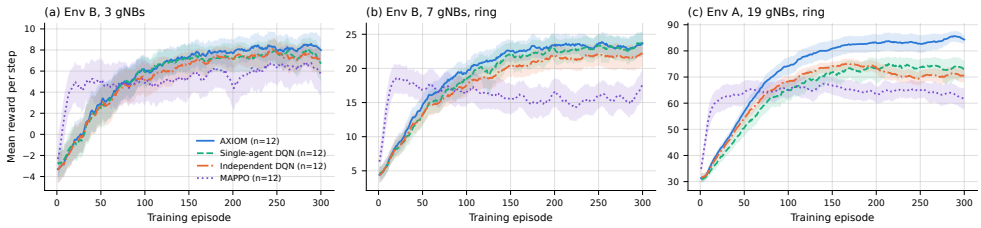


Fig. 11 Training curves, mean step reward per training episode over 12 seeds with 95% bands, smoothed over 10 episodes. a, b, Env B at three and seven gNBs. c, Env A at 19 gNBs on the ring layout.

on one CPU thread, well inside the 5 s control period of the live testbed. MAPPO is fastest at inference, because it scores all of a step's requests in one batch.

5.8 Reproducibility and Fresh-Seed Verification

We ran a verification campaign to check that the reported results reproduce and do not depend on the seeds chosen, with every pass criterion fixed before any run. First, we reran eleven reported cells on their original seeds in fresh processes. AXIOM in both environments and at nineteen gNBs, its MLP variant, MAPPO, the independent DQN, the tuned rules, federated training, and the dropout cell all reproduced their stored per-seed results exactly. The rule we rely on as the Env A oracle also matched the stored oracle exactly on all 30 seeds.

The single-agent DQN is the one exception to exact reproduction. Its training code builds the network before it sets the seed, so the first seed in each batch of runs starts from an unseeded initialisation, while later seeds in the batch reproduce exactly. Five reruns of one such seed earned 7.45 to 9.67 reward per step, against 8.74 stored. Dropping the affected seeds leaves every comparison with this baseline in the same direction. The one change is in Env A at three gNBs, where AXIOM's lead moves from +0.311 to +0.241 per step but now passes the Holm correction, at $p = 0.039$ against 0.066.

Second, we retrained every arm on fresh seeds from 3000 onwards, never used before, with 20 seeds at three gNBs, 15 in Env A at nineteen gNBs, and 10 in Env B at seven gNBs. All 26 predictions stated in advance for these fresh seeds held. In Env A at three gNBs AXIOM reached 98.2% of the oracle reward and beat the independent

Table 5 Model complexity. Parameters used at execution, greedy inference time per control step on one CPU thread, and median training time per seed.

Policy	Parameters, 3/7/19 gNBs	ms per step, 3/7/19 gNBs	Training s, 3/19 gNBs
AXIOM	7,782 / 7,782 / 7,782	1.04 / 1.98 / 4.79	624 / 892
AXIOM, MLP encoder	3,190 / 3,190 / 3,190	0.77 / 1.69 / 4.46	503 / –
Single-agent DQN	21,250 / 26,370 / 41,730	0.55 / 1.30 / 3.58	85 / 202
Independent DQN	55,302 / 129,038 / 350,246	0.59 / 1.38 / 3.73	181 / 1054
MAPPO	5,122 / 5,122 / 5,122	0.19 / 0.20 / 0.22	73 / 256

DQN and MAPPO at Holm-adjusted $p \leq 0.002$. At nineteen gNBs it kept 95.6% priority-slice compliance against 47.8% to 65.5% for the baselines, winning all six tests at Holm-adjusted $p \leq 0.001$.

In Env B at three gNBs AXIOM beat the static and accept-all rules in all 20 seeds and led every learned baseline at Holm-adjusted $p \leq 0.0002$. The tuned local rule again beat AXIOM, earning 9.175 against 8.782 per step, and at seven gNBs it beat every learned policy at $p \leq 0.004$. Two fresh-seed results are less favourable to AXIOM than the main campaigns. In Env A at three gNBs AXIOM won 11 of 20 seeds against the single-agent DQN but trailed it by 0.021 per step on average, at $p = 0.07$, so the pooled 40-seed lead over that baseline does not carry over to new seeds.

In Env B at seven gNBs the per-node MLP encoder beat graph attention by 1.38 reward per step, in 9 of 10 seeds at $p = 0.006$, which strengthens the finding that attention adds nothing. Third, a script written independently of the analysis pipeline, with its own log reader and Holm correction, recomputed from the raw logs the numbers in every results table and the key results sentences, including those of this subsection. All 206 checks pass after we corrected the three rounding errors it found, and all 24 quoted confidence intervals match a fresh bootstrap.

5.9 Live Deployment

This section reports what the real OAI testbed of Fig. 3, introduced in Section 3, can and cannot establish about the offline-trained policy. AXIOM itself is not deployed on the live testbed. Its multi-gNB policy needs at least two live gNBs, and our two-gNB attempt registered both gNBs with the core but did not sustain traffic, so a live AXIOM run is left to future work. This section therefore validates the deployment pipeline that AXIOM would use, namely the E2 control path, the PRB-ratio actuation, and real-time inference of an offline-trained checkpoint. For that we run a single-agent-DQN checkpoint, seed 900, against a real gNB, real User Equipments (UEs), and real per-slice User Datagram Protocol (UDP) traffic, at one UE per slice and again at two UEs per slice.

A capacity probe at a deliberately wide-open ceiling showed all three slices serving essentially 100% of offered demand with 0.000% loss at both loads. The frozen checkpoint's decision logic runs correctly end to end, through the real E2 control loop, with real accept and reject decisions, and its signature low-priority-only blocking of mMTC reappears at both loads, which shows that the deployment pipeline works end to end. Because the rig never actually runs short of capacity, the same pattern is equally consistent with the policy's trained priority bias, so we claim admission-control behaviour under contention only in the offline shared-budget environment, not live. Two integrity issues surfaced and were fixed during these checks. A scheduler-configuration error had turned the per-slice PRB-ratio ranges into raw grants below the gNB scheduler's hard five-PRB floor, which we fixed at the root, after which direct instrumentation confirmed zero below-floor grants across more than 100,000 readings. A separate unit mismatch in the live SLA-margin diagnostic was corrected by converting backlog into a queueing-delay estimate, using Little's law, against each slice's own latency budget.

6 Conclusion

This paper presented AXIOM, a multi-agent slice admission controller with a shared cluster encoder and decentralised per-slice action heads, together with an evaluation protocol designed to make its results trustworthy. It targets three gaps from our prior systematic review. It closes the first, trustworthy evaluation, tests the second, privacy-preserving coordination, and finds it still open, and validates only the control path of the third, live deployment.

In a static-load shared-budget environment, AXIOM comes within 1.1% of an oracle rule's reward at three gNBs and is the most reliable learned policy as the cluster grows. It keeps priority-slice compliance near 97% at seven and nineteen gNBs, while single-agent and independent DQNs and MAPPO fall to between 56% and 76%, across 36 per-layout comparisons over up to 30 seeds. In a harder environment with time-varying load and inter-cell coupling, AXIOM beats the static and accept-all rules and every learned baseline at three gNBs. A three-parameter rule tuned with domain knowledge still beats every learned policy there, however, and at seven gNBs AXIOM does no better than the DQN baselines. Graph attention added nothing measurable over a per-node encoder in either environment, so the gains come from the shared encoder, the per-slice heads, and joint training. Federated training showed no significant cost, while per-step DP-SGD gave no meaningful guarantee. These results are offline, and the live testbed validates the control path rather than AXIOM itself.

The evaluation safeguards changed what this paper finally reports. A trivial accept-all check exposed an earlier simulator in which admission control had nothing to do. A weak replication then exposed queue backlog that carried over between episodes, which had inflated AXIOM's earlier margins. Pilot runs on the harder environment showed that the original AXIOM learned only a static rule, and that our own collapse screen would have failed good policies there, and both were fixed in a pre-registration before any reported seed was run. The broader lesson is that learned admission control should be judged against trivial and tuned rules, with metrics tied to the objective, on independent seeds, under a pre-registered analysis.

Future work targets the gaps that this paper leaves open. The first is a live multi-gNB deployment of AXIOM itself. The second is closing the gap to tuned rules in coupled environments, for instance through longer training or by giving the learner the queue-threshold structure the rules exploit. The third is a privacy layer that adds noise to the per-round model updates rather than to every step, so that the formal guarantee becomes meaningful. We also plan Multi-Access Edge Computing (MEC) scaling, which adds compute-placement nodes to the cluster graph, cross-operator federation, and direct Quality of Experience (QoE) inference in the reward.

Acknowledgements. This work was supported by the Fundamental Research Grant Scheme (FRGS) of the Ministry of Higher Education (MOHE), Malaysia, under grant FRGS/1/2024/ICT06/TAYLOR/02/1.

Declarations

The following statements should be understood to apply to this manuscript unless otherwise indicated.

- **Funding:** This work was supported by the Fundamental Research Grant Scheme (FRGS), Ministry of Higher Education (MOHE), Malaysia, under grant number FRGS/1/2024/ICT06/TAYLOR/02/1.

- **Competing interests:** The authors declare no competing interests.
- **Ethics approval and consent to participate:** Not applicable, since this study involves no human participants, animal subjects, or identifiable patient data. All reported results are either offline simulation campaigns or measurements from a laboratory radio testbed.
- **Consent for publication:** Not applicable, since this manuscript contains no personal data or identifying information about any individual.
- **Data availability:** The experiment logs, checkpoints, and campaign result files underlying this paper's findings are available from the corresponding author on reasonable request.
- **Materials availability:** Not applicable, since this study involves no physical or biological materials.
- **Code availability:** The framework and experiment scripts used to produce this paper's results, including the reproducibility manifest and analysis scripts used to independently re-derive every table and figure in this paper, are available from the corresponding author on reasonable request.
- **Author contribution:** M. P. Kunasegran: conceptualisation, methodology, software, investigation, writing (original draft). W. L. Pang: supervision, writing (review and editing). S. K. Phang: supervision, writing (review and editing).

References

[1] Abadi M, Chu A, Goodfellow I, et al (2016) Deep learning with differential privacy. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp 308–318, https://doi.org/10.1145/2976749.2978318

[2] Ahmadi M, Moayyedi A, Sulaiman M, et al (2024) Generalizable 5G RAN/MEC slicing and admission control for reliable network operation. IEEE Trans Netw Service Manag https://doi.org/10.1109/TNSM.2024.3437217

[3] AlQahtani SA (2025) Dynamic slice allocation in 5G cloud RAN. Wireless Personal Communications 143(3-4):427–467. https://doi.org/10.1007/s11277-025-11835-y

[4] Bega D, Gramaglia M, Banchs A, et al (2020) A machine learning approach to 5G infrastructure market optimization. IEEE Transactions on Mobile Computing 19(3):498–512. https://doi.org/10.1109/TMC.2019.2896950

[5] Bouroudi A, Outtagarts A, Hadjadj-Aoul Y (2025) GNN-based multi-agent DRL for energy-efficient multi-domain 6G resource orchestration. In: Proc. IEEE Global Commun. Conf. (GLOBECOM), pp 4457–4462, https://doi.org/10.1109/GLOBECOM59602.2025.11432814

[6] Bun M, Steinke T (2016) Concentrated differential privacy: Simplifications, extensions, and lower bounds. In: Theory of Cryptography Conference (TCC), pp 635–658, https://doi.org/10.1007/978-3-662-53641-4_24

[7] Fatehi K, Rahmani Ghourtani M, Sonee A, et al (2026) Interpretable attention-based multi-agent PPO for latency spike resolution in 6G RAN slicing. In: Proc. IEEE Int. Conf. Commun. (ICC), https://doi.org/10.1109/ICC59461.2026.11586913

[8] Gong Y, Gong P, Jiang H, et al (2026) Intelligent resource allocation for multiaccess edge computing in 5G ultra-dense slicing network using federated multiagent DDPG. China Commun 23(1):273–289. https://doi.org/10.23919/JCC.fa.2022-0187.202601

[9] Kamal YB, Abdulkafi AA (2026) Reinforcement learning double deep Q-network techniques for NextG network. Wireless Personal Communications 146(6):3175–3197. https://doi.org/10.1007/s11277-026-12048-7

[10] Khani M, Jamali S, Sohrabi MK, et al (2024) Slice admission control in 5G cloud radio access network using deep reinforcement learning: A survey. International Journal of Communication Systems 37(13):e5857. https://doi.org/10.1002/dac.5857

[11] Krishnamoorthy R, Gireesh Babu CN, Gite P, et al (2026) Federated learning for dynamic resource allocation in 6G network slicing. Wireless Personal Communications https://doi.org/10.1007/s11277-026-11943-3

[12] Kulkarni D, Venkatesan M, Kulkarni AV (2024) Actor critic based reinforcement learning for joint resource allocation and throughput maximization in 5G RAN slicing. Wireless Personal Communications 138(1):623–640. https://doi.org/10.1007/s11277-024-11526-0

[13] Kunasegran MP, Pang WL, Phang SK (2025) Optimising resource allocation in 5G networks: Balancing URLLC and eMBB traffic under gNB congestion. In: 2025 Multimedia University Engineering Conference (MECON), pp 1–6, https://doi.org/10.1109/MECON67253.2025.11277096

[14] Kunasegran MP, Pang WL, Phang SK (2025) SLA-aware DRL for joint slice admission control and load balancing in multi-service 5G RAN. In: 2025 9th International Conference on Recent Advances and Innovations in Engineering (ICRAIE), pp 207–212, https://doi.org/10.1109/ICRAIE65839.2025.11239148

[15] Kunasegran MP, Pang WL, Phang SK (2026) Comprehensive review of optimisation techniques for user-centric distributed network slicing in 5G networks. IEEE Access 14:114630–114648. https://doi.org/10.1109/ACCESS.2026.3716911

[16] Li T, Sahu AK, Zaheer M, et al (2020) Federated optimization in heterogeneous networks. In: Proceedings of Machine Learning and Systems (MLSys), pp 429–450

[17] McMahan B, Moore E, Ramage D, et al (2017) Communication-efficient learning of deep networks from decentralized data. In: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), pp 1273–1282

[18] Mohajer A, Mirzaei A, Darabi M, et al (2026) Joint SLA-aware task offloading and adaptive service orchestration with graph-attentive multi-agent reinforcement learning. IEEE Trans Netw Service Manag 23:3737–3754. https://doi.org/10.1109/TNSM.2026.3673188

[19] Polese M, Bonati L, D'Oro S, et al (2023) ColO-RAN: Developing machine learning-based xApps for open RAN closed-loop control on programmable experimental platforms. IEEE Transactions on Mobile Computing 22(10):5787–5800. https://doi.org/10.1109/TMC.2022.3188013

[20] Polese M, Bonati L, D'Oro S, et al (2023) Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges. IEEE Communications Surveys & Tutorials 25(2):1376–1411. https://doi.org/10.1109/COMST.2023.3239220

[21] Qasim M, Imran M, Akram F, et al (2025) Transformer and graph neural network-based federated learning for cellular traffic prediction with sustainability analysis. IEEE Access 13:159649–159666. https://doi.org/10.1109/ACCESS.2025.3605380

[22] Rashid T, Samvelyan M, Schroeder de Witt C, et al (2020) Monotonic value function factorisation for deep multi-agent reinforcement learning. Journal of Machine Learning Research 21(178):1–51

[23] Raza MR, Natalino C, Öhlen P, et al (2019) Reinforcement learning for slicing in a 5G flexible RAN. Journal of Lightwave Technology 37(20):5161–5169. https://doi.org/10.1109/JLT.2019.2924345

[24] Shao Y, Li R, Hu B, et al (2021) Graph attention network-based multi-agent reinforcement learning for slicing resource management in dense cellular network. IEEE Trans Veh Technol 70(10):10792–10803. https://doi.org/10.1109/TVT.2021.3103416

[25] Sirma M, Kavak A, Inner AB (2025) Optimizing relay selection in D2D communication for next-generation wireless networks using multi-agent reinforcement learning: A novel approach. Wireless Personal Communications 140(3-4):945–969.

https://doi.org/10.1007/s11277-025-11753-z

[26] Sulaiman M, Moayyedi A, Ahmadi M, et al (2023) Coordinated slicing and admission control using multi-agent deep reinforcement learning. IEEE Trans Netw Service Manag 20(2):1110–1124. https://doi.org/10.1109/TNSM.2022.3222589

[27] Tashman DH, Cherkaoui S (2026) Adversarial attacks in AI-driven RAN slicing: SLA violations and recovery. In: Proc. Int. Wireless Commun. and Mobile Computing (IWCMC), https://doi.org/10.1109/IWCMC69287.2026.11580033

[28] Tsampazi M, D'Oro S, Polese M, et al (2023) A comparative analysis of deep reinforcement learning-based xApps in O-RAN. In: Proc. IEEE Global Communications Conference (GLOBECOM), pp 1638–1643, https://doi.org/10.1109/GLOBECOM54140.2023.10437367

[29] Veličković P, Cucurull G, Casanova A, et al (2018) Graph attention networks. In: International Conference on Learning Representations (ICLR)

[30] Yasin TV, Yu CM, Wang LC (2025) Differential privacy federated edge learning-assisted for securing RAN intelligent controller in O-RAN 6G communications. In: Proc. IEEE VTS Asia Pacific Wireless Commun. Symp. (APWCS), https://doi.org/10.1109/APWCS67981.2025.11151868

[31] Yu C, Velu A, Vinitsky E, et al (2022) The surprising effectiveness of PPO in cooperative multi-agent games. In: Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track

[32] Zangooei M, Golkarifard M, Rouili M, et al (2024) Flexible RAN slicing in Open RAN with constrained multi-agent reinforcement learning. IEEE Journal on Selected Areas in Communications 42(2):280–294. https://doi.org/10.1109/JSAC.2023.3336156

[33] Zhang H, Zhou H, Erol-Kantarci M (2022) Federated deep reinforcement learning for resource allocation in O-RAN slicing. In: Proc. IEEE Global Commun. Conf. (GLOBECOM), pp 958–963, https://doi.org/10.1109/GLOBECOM48099.2022.10001658